Final Project: Build a Python Application That Creates Value

The Challenge

Build a Python application that helps a specific person or group do something better. A useful project does not need to be a startup, make money, or solve a world-scale problem. It should create a clear benefit for its intended user: save time, reduce mistakes, make information easier to understand, improve access, support a decision, or make a task more enjoyable.

Think like a product team: understand the user, define the problem, build the smallest useful version, test whether it helps, and explain why someone would choose to use it. Your final application should be work you are proud to show in a professional portfolio and Git repository.

Start with Value, Not Features

Before choosing technology or asking AI to generate code, answer these questions:

1. **Who is the user?** Be more specific than "everyone." For example: a student planning assignments, a volunteer coordinator, or a small shop tracking supplies.
2. **What task or problem do they have?** Describe something they currently find slow, confusing, repetitive, error-prone, or difficult to access.
3. **What happens if the problem is solved?** Name the benefit from the user's point of view.
4. **How could you tell whether your program helped?** Choose a simple measure, such as time taken, errors avoided, tasks completed, or whether a user can find the needed information.
5. **What is the smallest useful first version?** Choose a few features that produce the benefit. Do not start with every idea you might eventually add.

Use this one-sentence value statement:

> For **[specific user]** who needs to **[task or need]**, my application **[what it does]** so they can **[valuable outcome]**. I will check whether it helps by **[simple measure or test]**.

Example: "For student club treasurers who need to track reimbursements, my application records requests and shows which are still pending so they can answer members quickly. I will test it with sample requests and check whether a treasurer can find the status of a request in under a minute."

In business, this is value: a useful outcome for a customer, employee, organization, or community. Value may be measured in time, fewer errors, clearer decisions, improved access, reduced waste, or satisfaction. You do not need to prove financial profit; you do need to explain who benefits and how.

Project Ideas with a Value Angle

These are starting points, not required topics. Make the audience and outcome your own.

Idea	User and possible value	Achievable first version	Possible extension
Campus club event planner	Club officers spend less time coordinating events and tracking RSVPs.	Create events, record sample RSVPs, and display an attendee list or count.	Add a calendar view or export a summary.

Idea	User and possible value	Achievable first version	Possible extension
Small-shop stock tracker	A shop worker can spot items that need reordering before they run out.	Add items, update quantities, and show a low-stock list.	Read and write a CSV file or chart stock changes.
Food pantry meal planner	A household can use ingredients already available and waste less food.	Enter sample ingredients and show matching meal ideas from a small local dataset.	Add dietary filters or a use-soon list. Avoid collecting sensitive health data.
Volunteer shift organizer	A coordinator can see uncovered shifts and reduce scheduling confusion.	Add sample shifts, assign volunteers, and report unfilled shifts.	Add conflict checks or a printable schedule.
Personal study planner	A student can see upcoming work and choose what to do next.	Add tasks with course, due date, and completion status; sort by date.	Add reminders or a progress summary.
Community resource finder	A user can find a relevant local service more quickly.	Search a small, verified sample dataset by category or keyword.	Add filters, accessibility information, or map links.
Trivia game with a clear audience	A host can run a themed game for a club, class, or community event.	Organize questions by category, run a round, and track scores.	Add team play, a question editor, or saved results.

A strong small application with a clear user benefit is better than a large project with many unfinished features. Avoid building only a thin wrapper around an AI chatbot; your application should have useful behavior, data, or workflow that you designed and can explain.

Worked Example: Stockroom Signals

This example shows how one project could grow from a business need into a useful Python application. It is a planning model, not a required topic or a complete solution to copy.

Start with the User and Value

User: A worker or owner at a small shop. **Problem:** Inventory and sales are in a spreadsheet, but it takes time to notice which items are running low or selling quickly. **Value statement:** "For a small-shop worker who needs to know what to reorder, Stockroom Signals reads a product and sales spreadsheet and highlights low-stock items and recent sales patterns so the worker can make a faster, more informed reorder list." **How to evaluate value:** Give a peer a small sample spreadsheet and ask them to find the three items most in need of attention. Record whether the app makes the answer easier to find than reading the raw rows.

This is business value: a useful outcome for a customer, employee, organization, or community. Value might mean time saved, fewer mistakes, a clearer decision, or improved access. You do not need to prove financial profit, but you should explain who benefits and how.

Define the First Useful Version

Start with a **synthetic CSV file** containing fictional records. The core version does not require a cloud account, real business records, or an AI service. It could:

1. Load a sample inventory/sales CSV and explain a missing or malformed file clearly.
2. Check required columns and report missing values or invalid quantities.

3. Calculate sales totals and identify stock below its reorder threshold.
4. Let the user sort or filter results and display a clear report.
5. Save a cleaned summary or reorder list to a new CSV file.

Possible columns are `item_id`, `item_name`, `category`, `quantity_on_hand`, `units_sold`, and `reorder_level`. Fictional example row: `P104,Notebook,Stationery,8,17,10`. Label the file as synthetic; do not use real shop records.

Outline the Program

Part	Possible Python work	User value
Load	Read CSV with the standard library or an approved data library.	Avoids manually retyping spreadsheet rows.
Validate	Check required columns, numeric quantities, and missing values.	Prevents misleading reports from bad input.
Transform	Compare stock to reorder level and summarize sales.	Turns raw rows into possible next actions.
Present	Show low-stock items, filters, and a concise summary.	Helps the user find what to review quickly.
Export	Write a reorder list or summary CSV.	Lets the user use results elsewhere.
Test	Try normal data, a missing file, invalid quantity, and no low-stock items.	Builds confidence that the tool behaves predictably.

Relevant course concepts could include functions, lists or dictionaries for records, loops and conditionals for checks, exceptions for file/conversion errors, and testing. Identify only the concepts you actually use.

Example Repository Outline

```
stockroom-signals/
|-- README.md
|-- requirements.txt          # only if third-party packages are used
|-- .gitignore
|-- data/
|   `-- sample_inventory.csv  # fictional data only
|-- src/
|   |-- main.py
|   |-- load_data.py
|   |-- validate_data.py
|   `-- reports.py
|-- tests/
|   `-- test_reports.py
`-- docs/
■`-- sample_report.png
```

This is an example, not a required framework; a smaller app can use fewer files. Make meaningful Git commits as you add a working slice, validation, report, tests, and documentation.

Build in Small Releases

1. Create the repository and README; commit the user need and sample-data description.
2. Load and display sample rows; commit a working first slice.
3. Add validation and friendly error handling; test invalid and missing input.
4. Add low-stock calculations and a report; compare with a hand-calculated example.
5. Ask a peer to follow the instructions and find items to review; improve any confusing step.
6. Add export, finish testing, document limitations, and prepare a short demo.

At each step ask: "Does this make the user's task easier or more reliable?" Move features that do not support the value statement to a future-work list.

Optional Advanced Extensions: Cloud Data, LLM APIs, and Agents

The core project must work locally with sample data and without a paid service, cloud account, or external AI API. Advanced integrations are optional and must not replace the core application. Get instructor approval before creating an account or using an external service. Use synthetic data only.

Spreadsheet to Snowflake to Python

If an approved no-cost or institution-provided Snowflake environment is available, students could:

1. Inspect the synthetic spreadsheet and identify its columns and data types.
2. Upload/load the sample file into the approved Snowflake environment.
3. Create or inspect a table schema; document column mapping and treatment of missing or invalid values.
4. Use SQL for a useful transformation, such as units sold by item or items below reorder level.
5. Connect from Python with an approved Snowflake connector and retrieve only the fields the app needs.
6. Compare results with the local CSV workflow or a hand-calculated test case.
7. Document setup and permissions, while keeping a local mode so reviewers can run the core app without cloud access.

This can demonstrate spreadsheet ingestion, schema design, SQL transformations, Python connectivity, and result validation. Product names, free access, and trial terms can change; verify current terms and approval before relying on them.

LLM API Feature

An LLM API can support a specific task, such as turning already-calculated inventory results into a short plain-language summary. Keep quantities, totals, and reorder rules in your own Python or SQL code; do not ask the model to invent numeric results. Document what information is sent, how output is checked, and what happens when the service is unavailable. Include a local or mocked fallback so the core app still runs.

AI Agent Feature

An **AI agent** uses a model to choose among steps or tools to pursue a goal. A bounded "inventory review assistant" might receive a request such as "What should I review for reordering?" and have access only to two read-only tools: `get_low_stock_items()` and `get_sales_summary()`. It could explain findings and point to the underlying data.

Keep the agent's role narrow and inspectable:

- It may read approved sample data and request specific read-only summaries.
- Python or SQL, not the model, performs calculations and enforces business rules.
- The agent proposes findings; a person decides whether to act.
- It must not place orders, send messages, change records, or access unrelated files/accounts.
- Show tool calls and results so a user can inspect how an answer was produced.
- Handle timeouts, invalid model output, unavailable services, and requests outside scope.
- Provide a non-agent report as fallback and test with mocked responses where possible.

Explain what the agent can access and do, how a person checks its output, and what you tested. An agent is not automatically more valuable than a regular program; its flexibility should justify its added cost, complexity, and risk.

Security, Cost, and Fair Access

- Never commit API keys or credentials. Read them from environment variables or an approved secret manager; document placeholder variable names, not real keys.
- Check pricing, rate limits, data retention, and service terms. Do not incur charges for the class project.
- Never send personal, confidential, or real business data to an AI model or cloud service.
- Provide a local/mock path so classmates and graders can test without accounts, credentials, internet access, or paid usage.
- Access to a particular provider or account must not determine whether a student can earn full core credit.

Optional Extension Credit

The core project can earn full credit without a cloud account, LLM API, or agent. Students may earn **up to 5 bonus points** for one instructor-approved external integration (such as Snowflake, an LLM API, or a bounded agent) that adds clear user value.

Bonus credit requires a working demonstration, clear setup documentation, secure credentials, useful failure/fallback behavior, and evidence that data or model output was checked. Account creation alone or an integration that only works on the student's machine does not earn bonus credit. The core application must remain usable without the service. The final score is capped at 100 points; equivalent local approaches are acceptable so provider access does not determine grades.

Proposal: Get Approval Before Building

Submit a short proposal with:

- Project name and the value statement above.
- Intended user and the task or problem.

- Three core features for the minimum useful version.
- One or two features you will deliberately leave out of the first version.
- At least three Python concepts from class that you expect to use, and where they may appear.
- Data and dependencies needed. Use sample or public non-sensitive data where possible.
- One test that checks the main feature and one test for an edge case.
- A brief note about risks: privacy, accuracy, accessibility, bias, permissions, or reliance on an external service.

Wait for instructor approval before building a project that needs paid services, credentials/API keys, private or sensitive data, substantial external infrastructure, or technologies outside the course. Do not put secrets or private personal information in code, prompts, screenshots, or Git.

Project Requirements

Your application must:

- Be a Python program that runs in the course-approved environment and has a clear intended user.
- Deliver at least three meaningful core features appropriate to the approved scope.
- Demonstrate at least three course concepts purposefully. Identify them in the README and point to where they appear in the code.
- Handle expected input and at least one relevant invalid-input, failure, or boundary case.
- Be tested with either automated tests where appropriate or a manual test checklist with actual results.
- Explain its value and how you evaluated whether the intended user benefit is achieved.
- Include a professional README with purpose, audience, value statement, features, setup, run instructions, usage, tests, limitations, and AI-use statement.
- Use Git throughout development with meaningful commits that show progress and decisions.
- Include dependency/setup information. Add `requirements.txt` if the project uses third-party Python packages; if it uses only the standard library, state the Python version and run command.

Use permitted assets, datasets, libraries, and services. Credit sources and follow their usage terms. Do not use real private data for a demonstration; use synthetic/sample data instead. Do not commit secrets, private information, virtual environments, or generated build output.

Plan, Build, and Learn with AI

AI is a development partner, not the product owner or the author of work you cannot explain. It can help you compare project scopes, clarify an error, suggest test cases, or review a small code section. Your decisions, verification, and understanding remain your responsibility.

Keep a short AI work log for meaningful help: the task, a summary of the prompt, what you accepted or changed, and how you checked the result. Summarize this in your README. Verify factual output and run suggested code. Never send passwords, API keys, private records, or other sensitive information to an AI service.

Example prompt for shaping an idea:

> I want to help [specific user] with [task/problem]. Ask me questions that will help me choose a small first version. Do not write code. Help me identify the user's benefit, three core features, what should be out of scope, and one way to test whether the program helps.

Example prompt for testing:

> Here is one feature and my current code. Suggest normal, boundary, and invalid-input tests. Explain why each test matters. Do not rewrite the feature for me.

Professional Git and Portfolio Expectations

Use the course-approved Git platform and visibility. Submit your repository link through Canvas as directed. A repository that the instructor cannot access may be considered unsubmitted until access is fixed; follow course directions for a private repository or approved backup.

Make the repository welcoming to a reviewer:

- Use a clear project name, organized files, and meaningful commit messages.
- Commit at natural milestones: setup, working feature, tests, documentation, and improvements.
- Write the README for someone who has never seen the project. State the user need and value before explaining implementation details.
- Include a screenshot or short demonstration media when useful and permitted.
- Document dependencies, sample data, setup, and how to run/test the program.
- Keep credentials and private information out of the repository.
- Credit external data, code, packages, and visual assets.

Milestones

The instructor will announce due dates. Expect to develop the project in these stages:

1. **Proposal:** user, problem, value statement, first-version features, course concepts, and risks.
2. **Plan:** pseudocode or flowchart, data plan, user interaction sketch, and test plan.
3. **First useful slice:** one end-to-end feature that gives the intended user a real benefit; commit it to Git.
4. **Feedback checkpoint:** demonstrate the current version, collect peer/instructor feedback, and choose a justified improvement.
5. **Feature-complete draft:** core features are implemented; tests and README are underway.
6. **Final release and demo:** tested application, professional repository, final README, and short presentation/reflection.

At each checkpoint, be ready to say what value the current version provides, what evidence supports that claim, and what you would improve next.

Final Submission Checklist

Submit the approved Git repository link through Canvas and any separately requested reflection or presentation materials. Before submission, confirm:

- A reviewer can identify the user, problem, and benefit quickly.
- The core features work and the application runs using the documented steps.
- Tests cover ordinary use and relevant edge cases, with results recorded.
- You explain how you evaluated user value and what you learned from feedback or testing.
- Git history shows steady work, not only one final upload.
- README, dependencies, data/asset sources, limitations, and AI assistance are documented.
- The repository contains no credentials or private information.
- You can demonstrate the application and explain the code and important decisions.

Grading Rubric (100 Points)

Area	Points	Full-credit evidence
User problem and value case	20	Specific audience and real need; clear benefit; realistic scope; a sensible way to evaluate whether the application helps.
Working application	20	Approved core features run and support the stated user task; interactions and output are understandable.
Course concepts and implementation	15	At least three course concepts are used appropriately; code is organized, readable, and explainable for this scope.
Testing, reliability, and iteration	15	Normal and edge cases are tested; results and limitations are honest; feedback or test findings lead to a thoughtful improvement.
Professional Git repository and setup	10	Meaningful commit history, clear organization, reliable run/setup steps, and appropriate dependency information.
README, demo, and portfolio presentation	10	Reviewer can understand the value, run or assess the application, and follow a clear demonstration.
AI transparency and reflection	10	Meaningful AI use is documented and checked; student explains decisions and reflects on what they learned.

Partial credit is based on demonstrated functionality, learning, and documented progress. A visually polished interface does not replace user value, working behavior, testing, or an explainable implementation.